



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Integrating UdeA's microgrid assets with OpenFMB.

Jhon Eduar Garcia

Daniel Ospina Acero
Jaime Alberto Vergara Tejada
Supervisores

Sergio Armando Gutiérrez Betancur
Coordinator
saguti@ieee.org

June 11, 2026

Contents

1	Integrating smart meters Schneider A9MEM3155.	3
1.1	Instantiate various meters in the OpenFMB adapter using the same YAML file.	5
1.2	Linking meters information from the adapter with HMI diagrams of OpenFMB	7
2	Integrating Modbus products (Power relays) with OpenFMB	12
2.1	Linking coils information from the adapter with HMI diagrams of OpenFMB	14
3	Integrating Color Control device with OpenFMB	17
3.1	Searching reading registers in the Color Control device to show them in the HMI of OpenFMB . . .	18
3.2	Searching writing registers in the Color Control device to modify them in HMI OpenFMB	24
4	Integrating Inverter Fronius with OpenFMB	27

1 Integrating smart meters Schneider A9MEM3155.

There is a single YAML file for all Modbus meters in the microgrid, which the adapter reuses via ‘Templating’. For this, first create the Modbus-master plugin; this file keeps the same ‘profile’ property for all meters, so all of them have the same stack of registers, then establish the other properties that will be changed among meters as ‘?’. This configuration is as shown in Figure 1.

```

◎ perseo-openfmb_l > perseo-openfmb_l_27_06 > modbus-master > modbus-meter.yaml > ...
1  name: medidor
2  log-level: Warn
3  adapter: 0.0.0.0
4  remote-ip: "?"
5  port: "?"
6  unit-identifier: "?"
7  response_timeout_ms: 1000
8  always-write-multiple-registers: false
9  auto_polling:
10   max_register_gaps: 0
11   max_bit_gaps: 0
12  heartbeats: []
13  file:
14   id: openfmb-adapter-template
15   edition: 2.0
16   version: 2.1.0.0
17   plugin: modbus-master
18  profiles:
19   - name: MeterReadingProfile
20     poll_period_ms: 1000
21     mapping:
22       readingMessageInfo:
23         messageInfo:
24           identifiedObject:
25             description:
26               value:
27                 string-field-type: ignored
28             mRID:
29               value:
30                 string-field-type: generated_uuid
31             name:
32               value:
33                 string-field-type: ignored
34             messageTimeStamp:
35               timestamp-field-type: message
36       meter:
37         conductingEquipment:
38           namedObject:
39             description:
40               value:
41                 string-field-type: ignored
42             name:
43               value:
44                 string-field-type: ignored
45             mRID:
46               string-field-type: primary_uuid
47               value: "?"
48       meterReading:

```

Figure 1: YAML file with 'Templating' for all Schneider A9MEM3155 meters in microgrid

1.1 Instantiate various meters in the OpenFMB adapter using the same YAML file.

After defining the Modbus-master plugin properties as mentioned before, you need to instantiate each meter in the adapter configurations using the same file, setting different values for the properties marked as ‘?’. Figure 2 shows the adapter configurations for adding a single Schneider meter, A9MEM3155.

```
perseo-openfmb_1 > perseo-openfmb_1_27_06 > config > adapter.yaml > {} plugins > {} modbus-master
6   logging:
10  rotating-file:

15  plugins:
16    capture:
17      enabled: false
18      file: capture.txt
19    replay:
20      enabled: false
21      file: capture.txt
22    dnp3-master:
23      enabled: false
24      thread-pool-size: 1
25      masters: []
26    dnp3-outstation:
27      enabled: false
28      thread-pool-size: 1
29      outstations: []
30  modbus-master:
31    enabled: true
32    thread-pool-size: 1
33    sessions:
34
35    #Medidor - Operador de Red
36    - path: modbus-master/modbus-meter.yaml
37      local-path: modbus-master/modbus-meter.yaml
38      session-name: SessionMeter1
39      overrides:
40      - key: remote-ip
41        value: 192.168.0.7
42      - key: port
43        value: 26
44      - key: unit-identifier
45        value: 1
46      - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
47        value: 00000001-0001-0020-0000-000000000001
48
```

Figure 2: Instance of a Schneider A9MEM3155 meters in the adapter configurations

In Figure 2, the configurations shown correspond to the ‘sessions’ property of the adapter’s Modbus-master plugin. This property allows the adapter to list any number of devices, each described in a YAML file. As illustrated in Figure 2, the YAML file can reuse a mapping file by specifying values for certain properties marked with a ‘?’ placeholder.

Specifically, the first meter instance shown in Figure 2 illustrates how the adapter can connect to a meter through an RS-485-to-Ethernet converter configured with ‘IP address’ 192.168.0.7 and ‘port’ 26. The slave number, set as ‘unit-identifier’ 1, is physically configured on the Schneider meter. For additional details, refer to the OpenFMB documentation.

Figure 3 presents an example where the adapter manages multiple instances of Schneider meters. Each instance uses distinct connection properties but shares the same register mapping defined in the file ‘modbus-master.yaml’.

```

◎ perseo-openfmb_I > perseo-openfmb_I_27_06 > config > adapter.yaml > {} plugins > {} modbus-master > [ ] sessions
15  plugins:
30    modbus-master:
32      thread-pool-size: 1
33      sessions:
34
35      #Medidor - Operador de Red
36      - path: modbus-master/modbus-meter.yaml
37        local-path: modbus-master/modbus-meter.yaml
38        session-name: SessionMeter1
39        overrides:
40        - key: remote-ip
41          value: 192.168.0.7
42        - key: port
43          value: 26
44        - key: unit-identifier
45          value: 1
46        - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
47          value: 00000001-0001-0020-0000-000000000001
48
49      #Medidor - Inversiones Quattro 15 kW
50      - path: modbus-master/modbus-meter.yaml
51        local-path: modbus-master/modbus-meter.yaml
52        session-name: SessionMeter2
53        overrides:
54        - key: remote-ip
55          value: 192.168.0.7
56        - key: port
57          value: 26
58        - key: unit-identifier
59          value: 2
60        - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
61          value: 00000001-0002-0020-0000-000000000001
62
63      #Medidor - Carga electrónica programable 1.5 kW
64      - path: modbus-master/modbus-meter.yaml
65        local-path: modbus-master/modbus-meter.yaml
66        session-name: SessionMeter3
67        overrides:
68        - key: remote-ip
69          value: 192.168.0.7
70        - key: port
71          value: 26
72        - key: unit-identifier
73          value: 3
74        - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
75          value: 00000001-0003-0020-0000-000000000001
76

```

Figure 3: Instantiate multiple meters in the OpenFMB adapter with the same YAML file

Figure 3 shows the adapter configuration to integrate 3 Schneider Meters. Each one has a different ‘session-name’ as they are different instances of the same modbus-master file. They also have different slave numbers but the same ‘IP address’ and ‘port’, which means that these 3 meters are physically connected to a same RS485 To Ethernet converter. In addition, each meter has a different mRID following the mRID codification established in Section 12 of OpenFMB documentation.

1.2 Linking meters information from the adapter with HMI diagrams of OpenFMB

After completing the adapter configurations as explained, you need to run the platform to access the OpenFMB HMI web-based environment. In the main view of HMI, the first step is to go to 'SETTINGS - Devices', as shown in Figure 4.

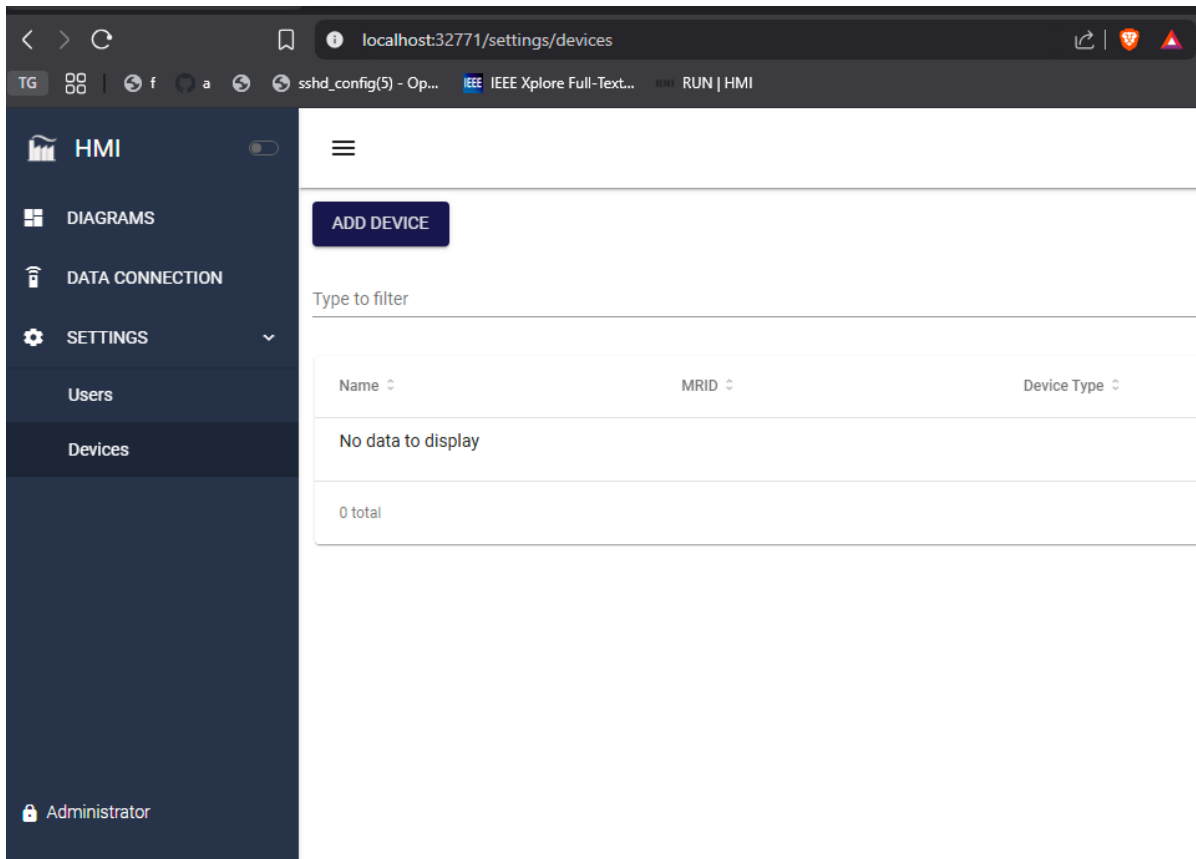


Figure 4: HMI view for linking a device name with an MRID

There, you can register a device by defining its name, mRID value, and device type (related to the profile to which that mRID belongs). In this way, you can add all the devices you need here, for example, three meters previously configured in the adapter's YAML file.

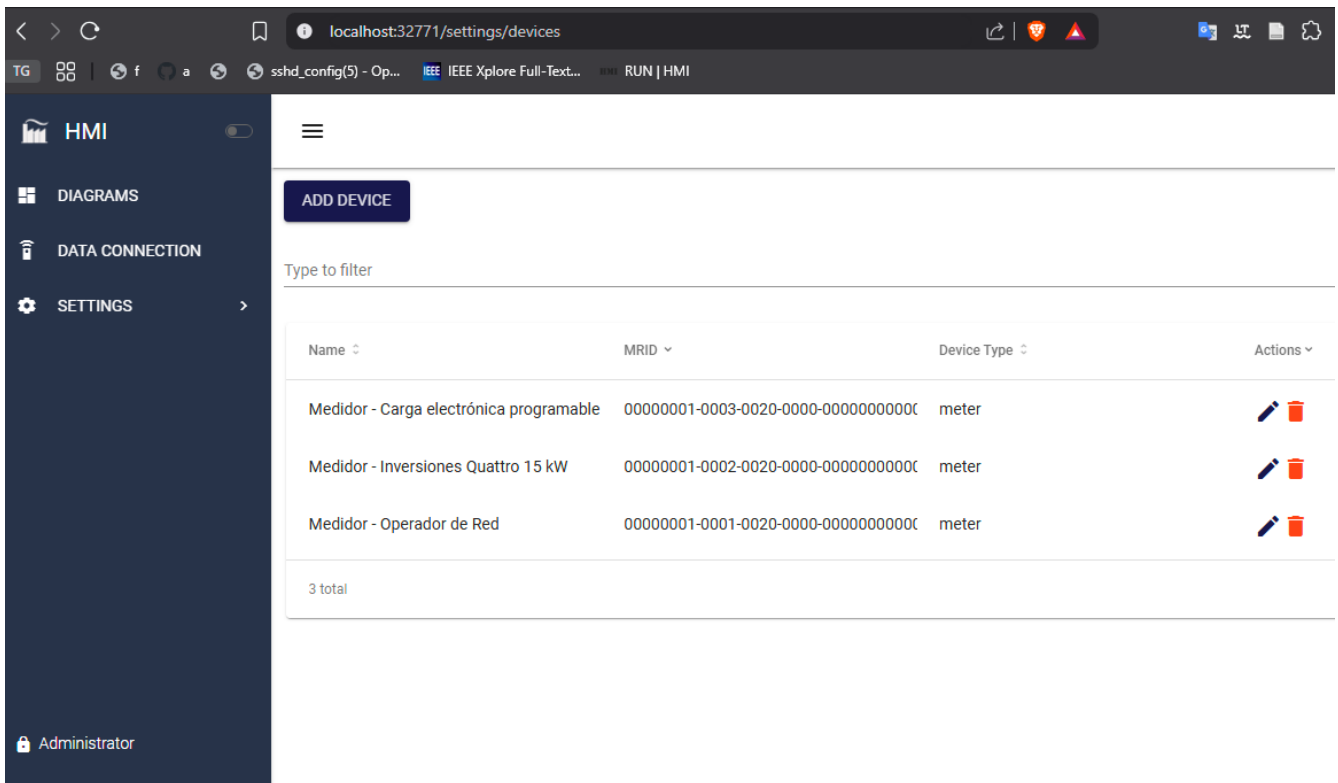


Figure 5: Register three Schneider meters with the mRIDs.

This first step enables the HMI to link the mRID with a device or register name. In addition, it is important to note that the device type is set to 'meter', since the profiles corresponding to these mRIDs only map reading registers. This means that the adapter sends this data to the OpenFMB HMI through the standardized bus.

The next step is to create a new diagram that contains 'Measure Box' icons, which link an MRID and its mapped registers. At first, enter 'DIAGRAMS - ADD DIAGRAM' and set its name, then, in 'DIAGRAMS' view, select the 'Design' option of the created diagram. These steps are shown in Figures 6 and 7.

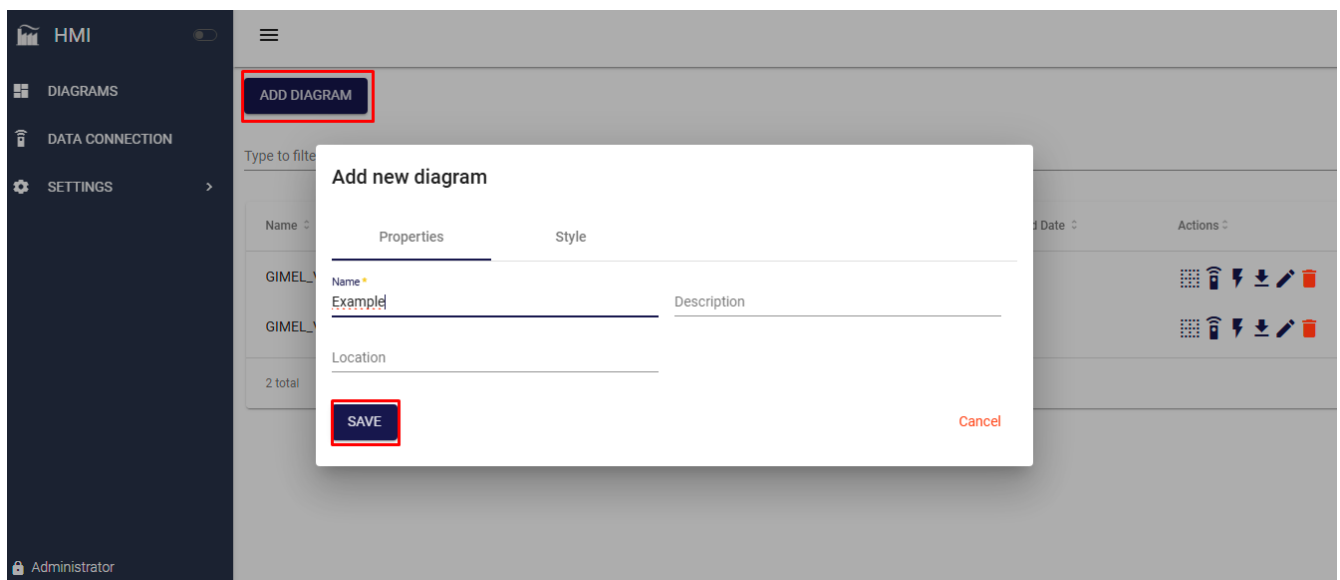


Figure 6: Creating an Example diagram in HMI

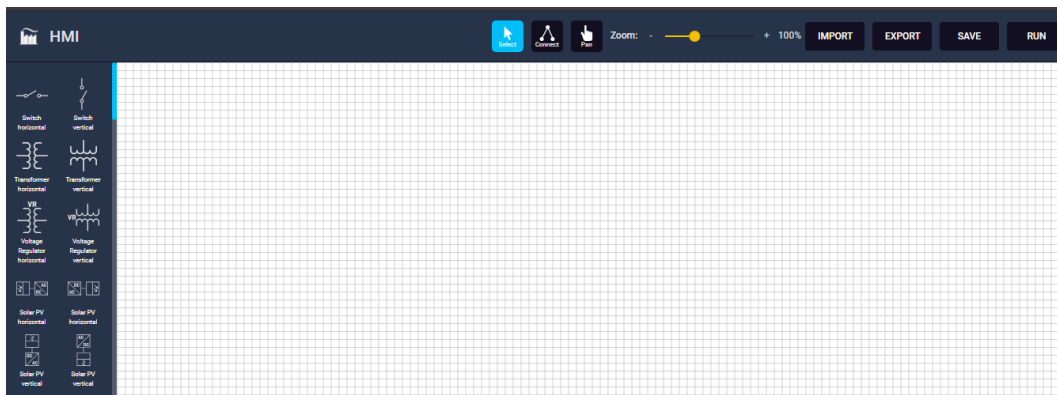


Figure 7: Empty diagram

To continue, scroll down in the left panel and drag the ‘Measure Box’ icon to the diagram. Then, select a device to link with this icon, click ‘Done’, and finally click the ‘SAVE’ button in the upper-right menu. Repeat these steps to integrate three meters into the diagram, as shown in Figures 8 and 9. Each ‘Measure Box’ icon has a label field that can be used to name the icon optionally.

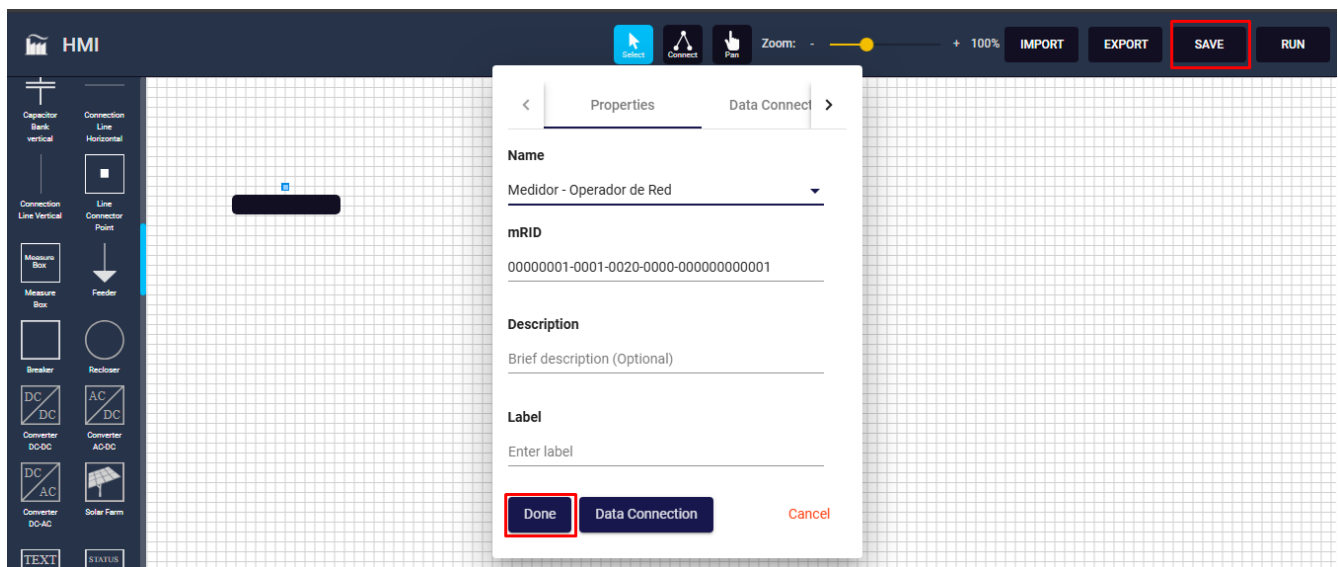


Figure 8: Adding a ‘Measure Box’ icon and linking it with an mRID

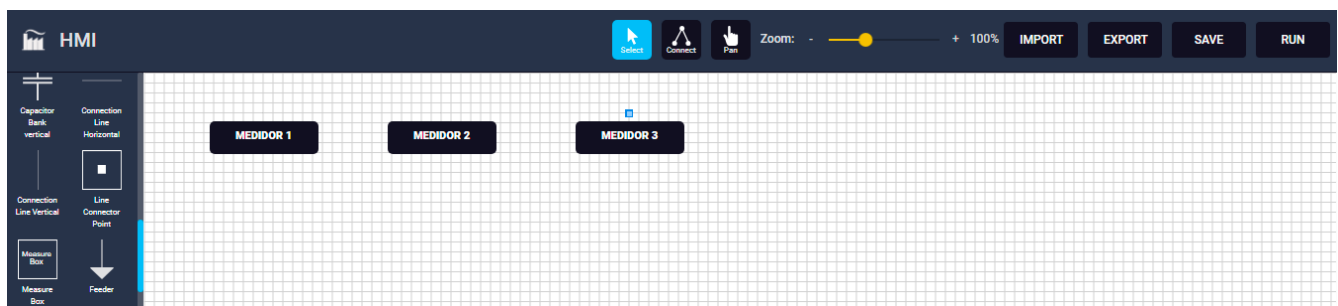
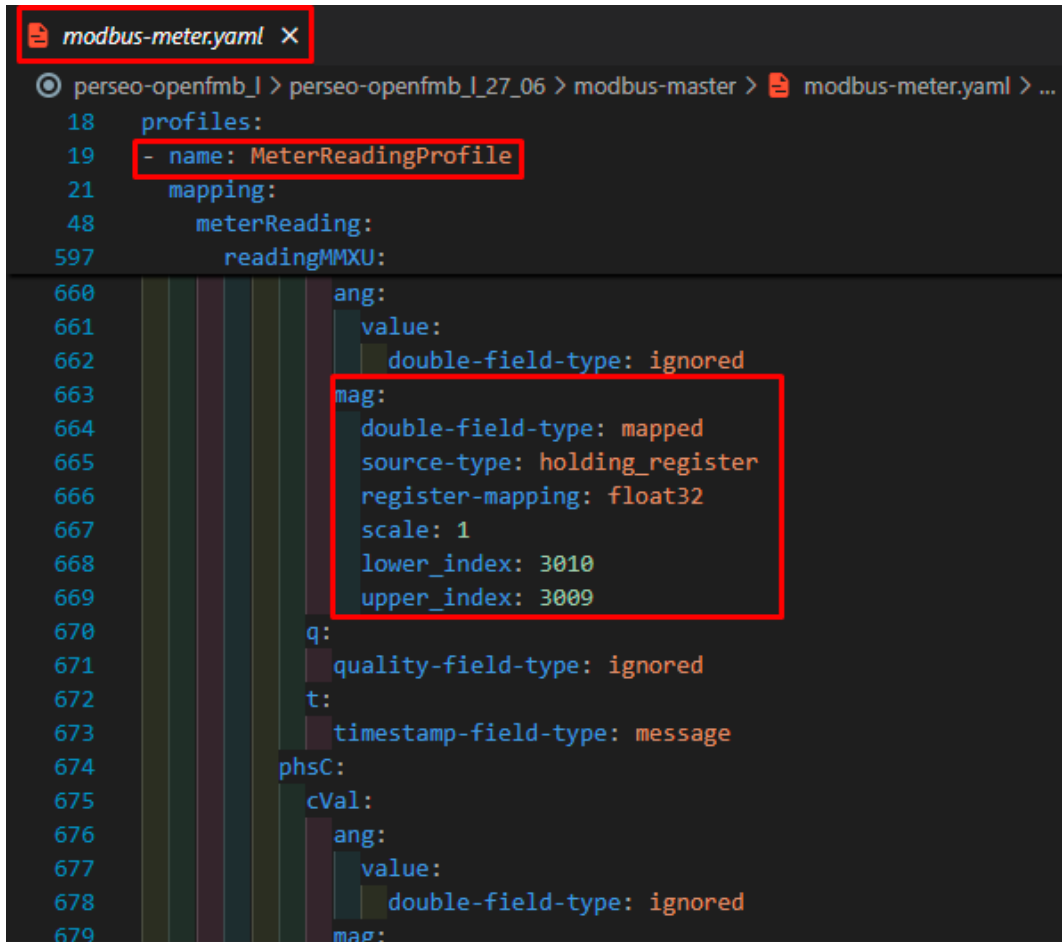


Figure 9: Diagram with three ‘Measure Box’ for displaying meter’s data

After linking a 'Measure Box' icon with an MRID, it's time to establish the specific map of the profile field to this icon, so first click on 'Measure Box' and then click on the 'Data Connection' button.

'Data Connection' view allows the user to select specific fields from an OpenFMB profile and associate them with the 'Measure Box' icon for monitoring. The selected field corresponds to the register you want to read in the 'Measure Box'. Remember that each MeterReadingProfile for A9MEM3155 meters contains many registers. Figures 10, 11, and 12 show how to link specific fields of a MeterReadingProfile to a Measure Box icon, based on the register you want to monitor.



```
modbus-meter.yaml X
perseo-openfmb_l > perseo-openfmb_l_27_06 > modbus-master > modbus-meter.yaml > ...
18 profiles:
19 - name: MeterReadingProfile
21 mapping:
48 meterReading:
597 readingMMXU:
660 ang:
661 value:
662 double-field-type: ignored
663 mag:
664 double-field-type: mapped
665 source-type: holding_register
666 register-mapping: float32
667 scale: 1
668 lower_index: 3010
669 upper_index: 3009
670 q:
671 quality-field-type: ignored
672 t:
673 timestamp-field-type: message
674 phsC:
675 cVal:
676 ang:
677 value:
678 double-field-type: ignored
679 mag:
```

Figure 10: Path for map register 3010 with a MeterReadingProfile

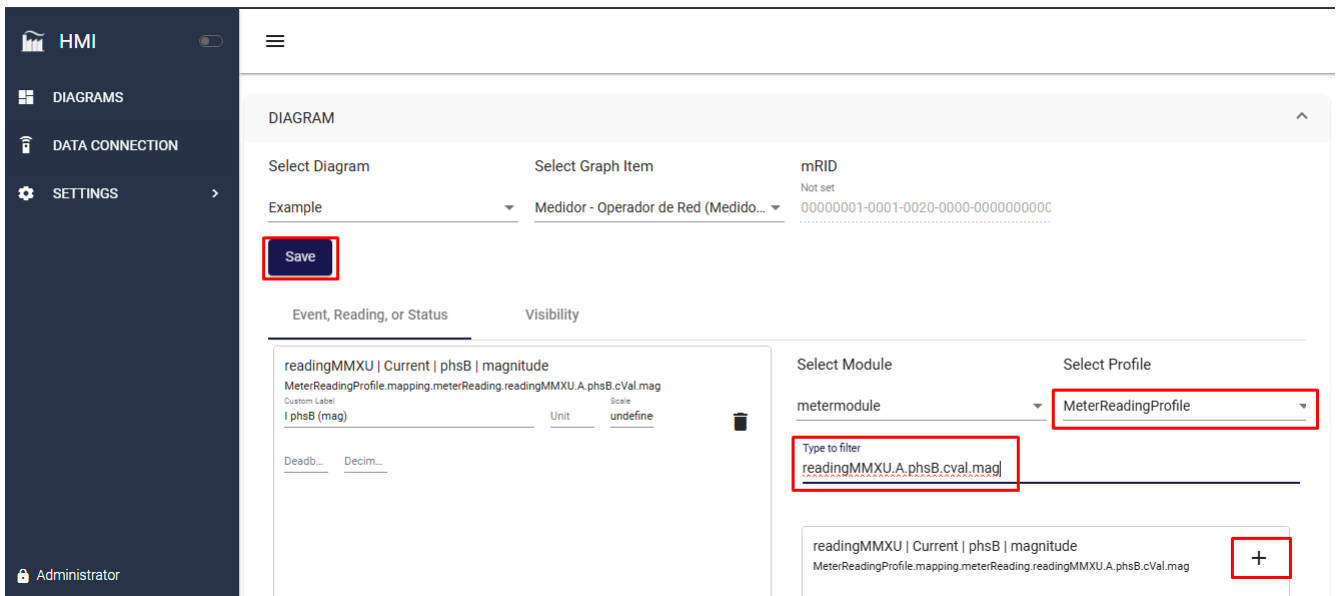


Figure 11: Usage of 'Data Connection' view to link the data path from a MeterReadingProfile to a 'Measure Box' item

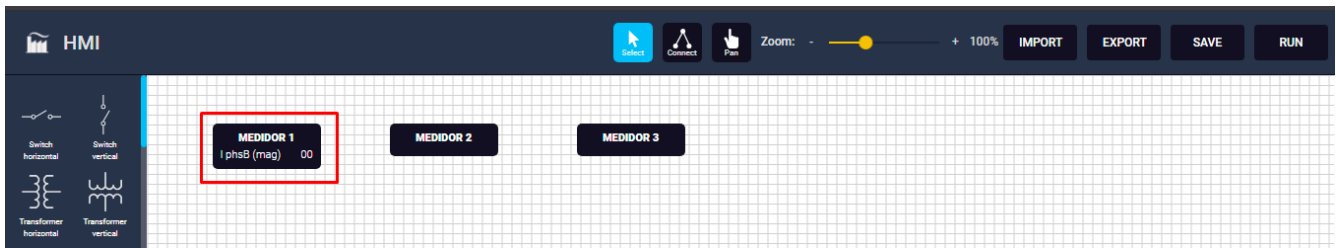


Figure 12: Visualization of a 'Measure Box' item linked with a data path of a MeterReading Profile

Figure 10 shows the specific field of the MeterReadingProfile used to map register 3010, which corresponds to the average current of the A9MEM3155 meter. The same process applies to the other 19 registers mapped with this profile. This field is then selected in the 'Data Connection' view and added to the Reading section of the 'Measure Box' icon (Figure 11). After clicking the Save button in the Data Connection view, the result appears as shown in Figure 12.

If everything is configured correctly, clicking the RUN option in the upper-right menu will display the A9MEM3155 meter's real-time measurements captured by the adapter on the HMI.

Following these steps, Figure 13 shows the monitoring of a set of registers for three Schneider A9MEM3155 meters.

- Average Current Register 3010: MeterReadingProfile.mapping.meterReading.readingMMXU.A.phsB.cVal.mag
- Average L-N Voltage Register 3036: MeterReadingProfile.mapping.meterReading.readingMMXU.PhV.net.cVal.mag
- Total Active Power register 3060: MeterReadingProfile.mapping.meterReading.readingMMXU.PhV.phsC.cVal.mag
- Total Reactive Power register 3068: MeterReadingProfile.mapping.meterReading.readingMMXU.PPV.phsAB.cVal.mag
- Total apparent Power register 3076: MeterReadingProfile.mapping.meterReading.readingMMXU.PPV.phsBC.cVal.mag
- Power Factor register 3084: MeterReadingProfile.mapping.meterReading.readingMMXU.PPV.phsCA.cVal.mag
- Frequency register 3110: MeterReadingProfile.mapping.meterReading.readingMMXU.VA.net.cVal.mag



Figure 13: Real-time monitoring of Schneider meters with OpenFMB HMI

2 Integrating Modbus products (Power relays) with OpenFMB

The Modbus products used in this microgrid have 12 coils. These coils are the variables that need to be mapped using the adapter's Modbus-master plugin. Figure 14 shows the proposed mapping for this device. If other Modbus products are needed, this single file can be reused.

```
modbus-boxes.yaml X
perseo-openfmb_1 > perseo-openfmb_1_27_06 > modbus-master > modbus-boxes.yaml >
1  name: modbox
2  log-level: Warn
3  adapter: 0.0.0.0
4  remote-ip: "?"
5  port: "?"
6  unit-identifier: "?"
7  response_timeout_ms: 1000
8  always-write-multiple-registers: false
9  auto_polling:
10   max_register_gaps: 0
11   max_bit_gaps: 0
12  heartbeats: []
13  file:
14   id: openfmb-adapter-template
15   edition: 2.0
16   version: 2.1.0.0
17   plugin: modbus-master
18  profiles:
19  > - name: SwitchStatusProfile ...
173 > - name: SwitchStatusProfile ...
327 > - name: SwitchStatusProfile ...
481 > - name: SwitchStatusProfile ...
635 > - name: SwitchStatusProfile ...
789 > - name: SwitchStatusProfile ...
943 > - name: SwitchStatusProfile ...
1097 > - name: SwitchStatusProfile ...
1251 > - name: SwitchStatusProfile ...
1405 > - name: SwitchStatusProfile ...
1559 > - name: SwitchStatusProfile ...
1713 > - name: SwitchStatusProfile ...
1867
1868 > - name: SwitchDiscreteControlProfile ...
1964 > - name: SwitchDiscreteControlProfile ...
2060 > - name: SwitchDiscreteControlProfile ...
2156 > - name: SwitchDiscreteControlProfile ...
2252 > - name: SwitchDiscreteControlProfile ...
2348 > - name: SwitchDiscreteControlProfile ...
2444 > - name: SwitchDiscreteControlProfile ...
2540 > - name: SwitchDiscreteControlProfile ...
```

Figure 14: YAML file for mapping the coils of a Modbus product

As illustrated in Figure 14, some properties in the file can change between Modbus products: ‘remote IP’, ‘port’, and ‘unit identifier’. Then there is the ‘profiles’ property; the first 12 profiles are SwitchStatusProfile, which are used in the HMI to show the state of a coil (the use of status profiles is explained below). The remaining 12 profiles are SwitchDiscreteControlProfile, which enable the adapter and HMI to write the value to a coil.

The following figure shows how to instantiate a single Modbus product in the OpenFMB adapter’s YAML file.

```
adapter.yaml X
perseo-openfmb_l > perseo-openfmb_l_27_06 > config > adapter.yaml > {} plugins > {} modbus-master >
15  plugins:
30    modbus-master:
31      enabled: true
32      thread-pool-size: 1
33      sessions:
34
35    # Productos Modbox
36
37    # Modbox - Master
38    - path: modbus-master/modbus-boxes.yaml
39      local-path: modbus-master/modbus-boxes.yaml
40      session-name: SessionBox1
41      overrides:
42        - key: remote-ip
43          value: 172.26.0.1
44        - key: port
45          value: 5050
46        - key: unit-identifier
47          value: 2
48        - key: profiles[0].mapping.protectedSwitch.conductingEquipment.mRID.value
49          value: 00000002-0001-1001-0000-000000000001
```

Figure 15: Instance of a Modbus product in the adapter's file configuration

Figure 15 illustrates how to instantiate a Modbus product. However, the mRID is only set for the first Switch-StatusProfile. The remaining 23 must be established, considering that each status profile has the same mRID as a DiscreteControl profile. This means that each Modbus product has 12 mRIDs, with each one enabling access to the status and writing functions. The process for instantiating additional Modbus products is the same as explained in Section 1.1.

2.1 Linking coils information from the adapter with HMI diagrams of OpenFMB

With OpenFMB running, go to 'Settings - Devices' and register the mRID of the coils to be controlled within the diagram. Figure 16 shows what it looks like.

 HMI

 DIAGRAMS

 DATA CONNECTION

 SETTINGS

Users

Devices

ADD DEVICE

Type to filter

Name	MRID	Device Type	Actions
Producto Modbus Master - C3	00000002-0001-1001-0000-000000000003	switch	 
Producto Modbus Master - C4	00000002-0001-1001-0000-000000000004	switch	 
Producto Modbus Master - C5	00000002-0001-1001-0000-000000000005	switch	 
Producto Modbus Master - C6	00000002-0001-1001-0000-000000000006	switch	 
Producto Modbus Iluminación - C1	00000002-0002-1001-0000-000000000001	switch	 
Producto Modbus Iluminación - C2	00000002-0002-1001-0000-000000000002	switch	 
Producto Modbus Iluminación - C3	00000002-0002-1001-0000-000000000003	switch	 
Producto Modbus Terraza - C1	00000002-0003-1001-0000-000000000001	switch	 
Producto Modbus Terraza - C2	00000002-0003-1001-0000-000000000002	switch	 
Producto Modbus Terraza - C3	00000002-0003-1001-0000-000000000003	switch	 

Figure 16: Linking the coils' name with mRIDs of a Modbus product

After registering the mRIDs, go to a diagram and drag the switch icon labeled 'Button'. Then, assign an mRID to this new icon. Remember that a single icon controls a single coil. Finally, click the 'Save' option in the top-right menu.

Next, go to the 'Data Connection' section by clicking on the 'Button' icon, as shown in Figure 17. Then, link the field from the status profile to the 'Event, Reading or Status' section. Also, link the field from the DiscreteControl profile to the 'Control' section. Finally, click on 'Save'; these steps are shown in Figures 18 and 19.

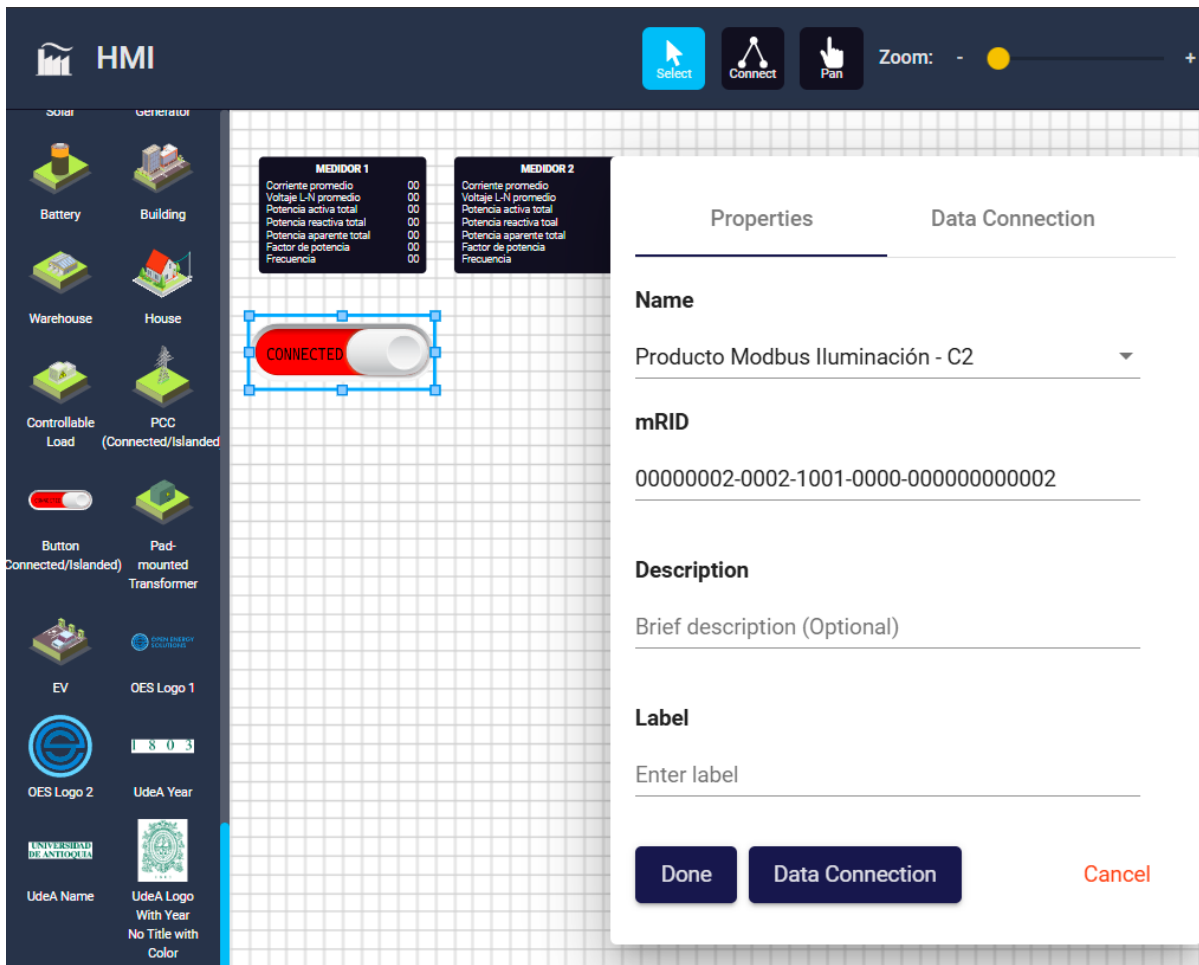


Figure 17: Linking a switch 'Button' with an mRID of a Modbus product's coil

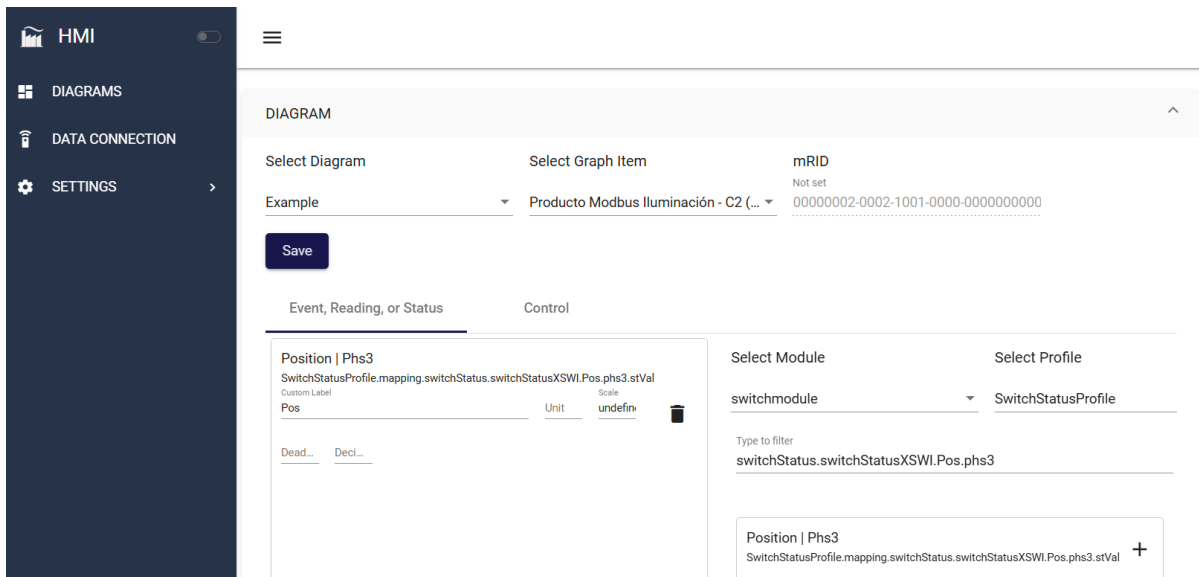


Figure 18: Linking the status data path to the switch 'Button' in 'Data Connections' view

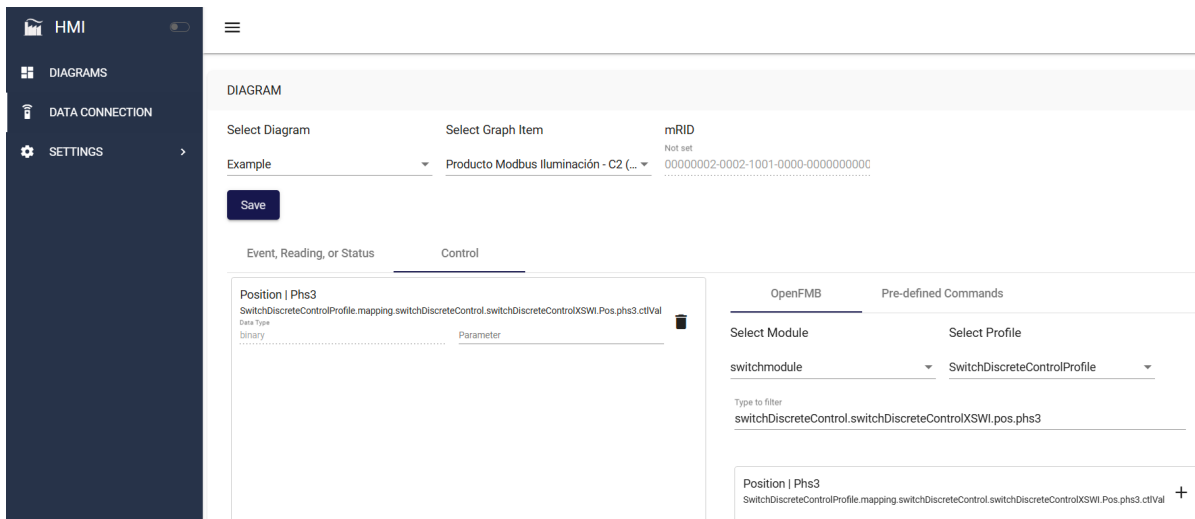


Figure 19: Linking the control data path to the switch 'Button' in 'Data Connections' view

After linking the Status and Control data paths with the switch 'Button' that is also linked to its mRID, you can run the diagram and modify the state of the mapped coil. You can also see the coil state in the switch button color. An example diagram shows these steps running in Figure 20.

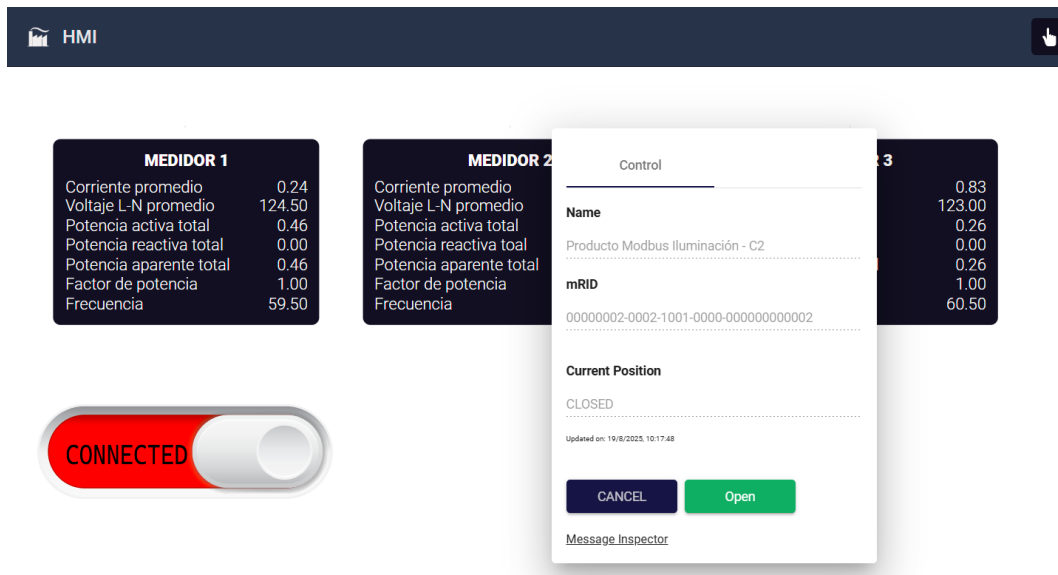


Figure 20: Example diagram with coil control.

3 Integrating Color Control device with OpenFMB

The Color Control is a device capable of managing other Intelligent Electronic Devices (IEDs) by sharing registers with them as a tree internal connection. Since it has four slaves (100, 225, 245, and 246), it contains a large number of reading and writing registers. As a strategy to simplify this complexity, Python is used to generate a separate YAML file for each slave, grouping all the corresponding registers in an organized manner, taking as input all the registers' data, for example, in CSV format.

The following provides a brief explanation of how to integrate a single register from any of the Color Control slaves. In practice, there are four YAML files, one for each slave, and the registers are distributed among them as described below.

- Slave 100: 35 reading registers and 21 writing registers.

- Slave 225: 88 reading registers and 1 writing register.
- Slave 245: 54 reading registers and 1 writing register.
- Slave 246: 68 reading registers and 61 writing registers.

Moreover, a single MeterReadingProfile groups 15 reading registers, while each writing register is associated with a single DiscreteControlProfile (either Resource or Switch). To assign the mRID, we applied the following codification strategy:

- 8 digits: unique consecutive for Color Control
- 4 digits: number of slaves of the color Control.
- 4 digits: number of registers grouped in that profile.
- 4 digits: zero sequence.
- 12 digits: counting of reading profiles for a single slave or the address number of the writing register.

To instantiate the Color Control device in the adapter's configuration, you need to set up the four slave files. The detailed mapping files for all the slaves are described in the OpenFMB project repository under the names: 'modbus-master-CCslave100.yaml', 'modbus-master-CCslave225.yaml', 'modbus-master-CCslave245.yaml', and 'modbus-master-CCslave246.yaml'.

This project repository also includes support files that organize the data path of each register and the specific mRID to be linked with an HMI icon for either reading or writing. The following section illustrates the procedure for identifying both the mRID and the corresponding data path for a reading operation.

3.1 Searching reading registers in the Color Control device to show them in the HMI of OpenFMB

The reading registers 840 and 841 of the Color Control contain, respectively, the voltage and current of a battery system connected to the Color Control through a Victron Quattro inverter/charger. In the file 'lectura-unificada-CC.xlsx' of the project repository (main branch - scripts - Esquemas de Identificacion), it is possible to find all the reading registers of Color Control with its slave number, mRID, and data path. Figure 21 illustrates the search of registers 840 and 841.

	A	B	C	D	E	F
1	slave	mRID	Indice	Variable	Address	Type
2	100	00000006-0100-0015-0000-000000000001	0	VA, net	808	uint16
3	100	00000006-0100-0015-0000-000000000001	0	VA, neut	809	uint16
4	100	00000006-0100-0015-0000-000000000001	0	VA, phsA	810	uint16
5	100	00000006-0100-0015-0000-000000000001	0	VA, phsB	811	uint16
6	100	00000006-0100-0015-0000-000000000001	0	VA, phsC	812	uint16
7	100	00000006-0100-0015-0000-000000000001	0	VAr, net	813	uint16
8	100	00000006-0100-0015-0000-000000000001	0	VAr, neut	814	uint16
9	100	00000006-0100-0015-0000-000000000001	0	VAr, phsA	815	uint16
10	100	00000006-0100-0015-0000-000000000001	0	VAr, phsB	816	uint16
11	100	00000006-0100-0015-0000-000000000001	0	VAr, phsC	817	uint16
12	100	00000006-0100-0015-0000-000000000001	0	W, net	818	uint16
13	100	00000006-0100-0015-0000-000000000001	0	W, neut	819	uint16
14	100	00000006-0100-0015-0000-000000000001	0	W, phsA	820	int16
15	100	00000006-0100-0015-0000-000000000001	0	W, phsB	821	int16
16	100	00000006-0100-0015-0000-000000000001	0	W, phsC	822	int16
17	100	00000006-0100-0015-0000-000000000002	1	VA, net	823	int16
18	100	00000006-0100-0015-0000-000000000002	1	VA, neut	824	int16
19	100	00000006-0100-0015-0000-000000000002	1	VA, phsA	825	int16
20	100	00000006-0100-0015-0000-000000000002	1	VA, phsB	826	int16
21	100	00000006-0100-0015-0000-000000000002	1	VA, phsC	830	uint64
22	100	00000006-0100-0015-0000-000000000002	1	VAr, net	840	uint16
23	100	00000006-0100-0015-0000-000000000002	1	VAr, neut	841	int16
24	100	00000006-0100-0015-0000-000000000002	1	VAr, phsA	842	int16

Figure 21: Finding information about reading registers in support files

In Figure 21, the file 'lectura-unificada-CC.xlsx' matches a reading register with its type (16 bits or 32 bits), the data path, the profile index in adapter configuration, and its slave number. For 840 and 841, this is the information:

- slave number: 100.
- mRID: 00000006-0001-0015-0000-000000000002.
- Index: 1.
- Data path: VAr - net, VAr - neut.
- type: uint16, int16

With this information, you can receive the data from the adapter to show it in an HMI diagram. Also, listed information allows you to verify and modify the properties of register's mapping, these verifications are located in the YAML files of the adapter and modbus-master plugin. This verification is as follows.

```
adapter.yaml X
perseo-openfmb_l > perseo-openfmb_l_27_06 > config > adapter.yaml > {} plugins > {} modbus-master > [ ] sessions > {} 0 > [ ] overrides > {} 1 > value
15  plugins:
30  modbus-master:
32    thread-pool-size: 1
33    sessions:
34
35  # Color control
36  #####
37  - path: modbus-master/modbus-master-CC_slave_100.yaml
38    local-path: modbus-master/modbus-master-CC_slave_100.yaml
39    session-name: Session_slave_100
40    overrides:
41      - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
42        value: 00000006-0100-0015-0000-000000000001
43      - key: profiles[1].mapping.meter.conductingEquipment.mRID.value
44        value: 00000006-0100-0015-0000-000000000002
45      - key: profiles[2].mapping.meter.conductingEquipment.mRID.value
46        value: 00000006-0100-0005-0000-000000000003
47      - key: profiles[3].mapping.conductingEquipment.mRID.value
48        value: 00000006-0100-1001-0000-000000000806
```

Figure 22: Identification of the specific profile that contains registers 840 and 841

Figure 22 shows the profile with the mRID from the support files of the OpenFMB project repository, the same profile that has the mapping for 840 and 841. This profile, the one with index 1, is the second MeterReadingProfile for the slave 100.

```
modbus-master-CC_slave_100.yaml X
perseo-openfmb_l > perseo-openfmb_l_27_06 > modbus-master > modbus-master-CC_slave_100.yaml > [ ] profiles > {} 1 >
20  profiles:
1060  - name: MeterReadingProfile
1062    mapping:
1089      meterReading:
1638      readingMMXU:

1871 > VA: ...
1948  VAr:
1949    net:
1950      cVal:
1951        ang:
1952          value:
1953            double-field-type: ignored
1954          mag:
1955            double-field-type: mapped
1956            index: 839
1957            register-mapping: uint16
1958            scale: 10.0
1959            source-type: holding_register
1960        q:
1961          quality-field-type: ignored
1962        t:
1963          timestamp-field-type: ignored
1964      neut:
1965        cVal:
1966          ang:
1967            value:
1968              double-field-type: ignored
1969            mag:
1970              double-field-type: mapped
1971              index: 840
1972              register-mapping: sint16
1973              scale: 10.0
1974              source-type: holding_register
1975        q:
1976          quality-field-type: ignored
1977        t:
1978          timestamp-field-type: ignored
```

Figure 23: Data paths of 840 and 841 registers into the slave 100 mapping

Figure 23 illustrates the data path of the registers. Then, with the mRID and the data paths, it's time to add a new icon in an HMI diagram, enabling real-time monitoring. This step is shown in the figure below.

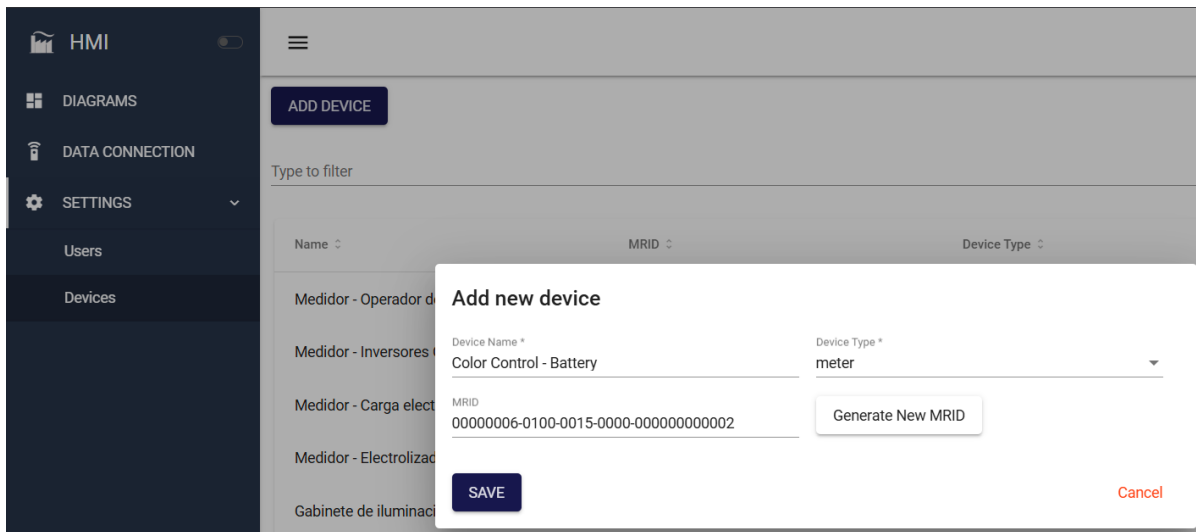


Figure 24: Adding a mRID to monitor specific registers from Color Control

After adding the mRID, the next step is to assign it to a new 'Measure Box'.

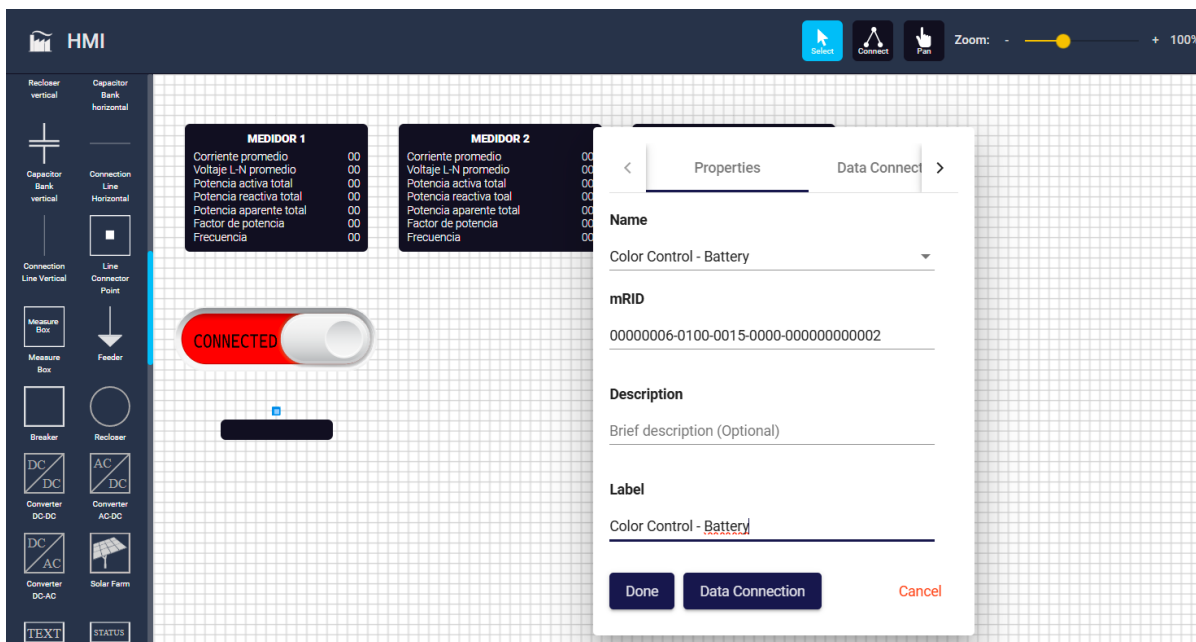


Figure 25: Linking a 'Measure Box' with the mRID of 840 and 841 registers of Color Control's slave.

The last step is linking the data paths to the 'Measure Box' in the 'Data Connection' view of HMI. This step is shown in the figure below.

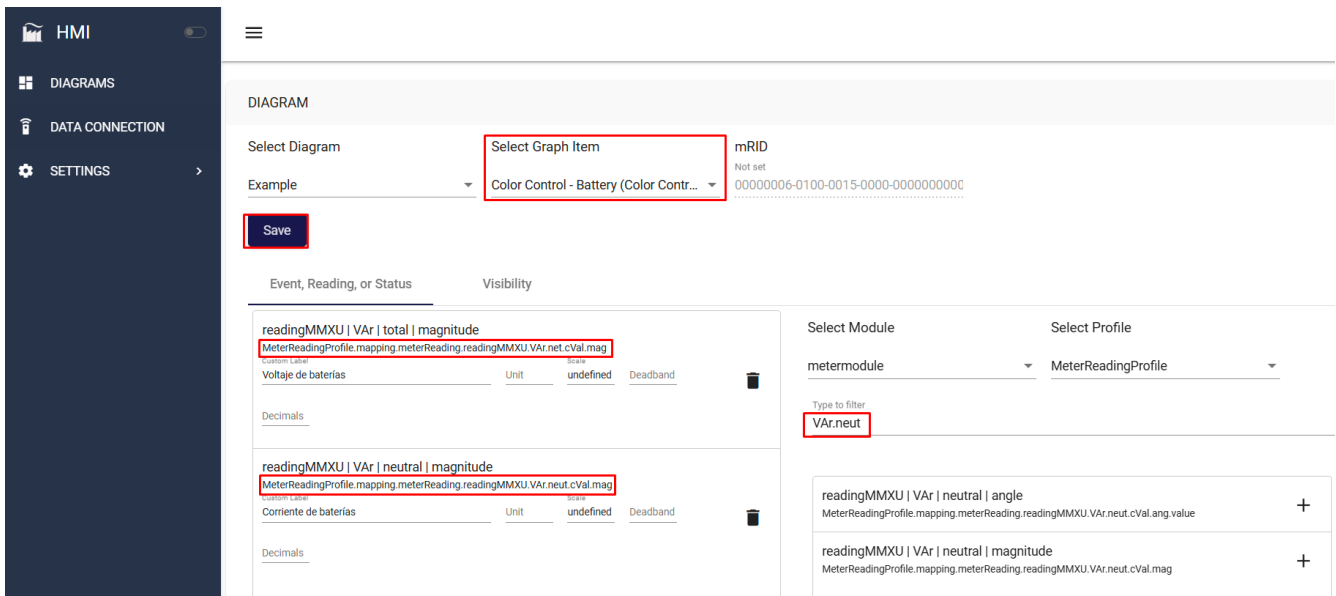


Figure 26: Linking data paths of the registers with a 'Measure Box'.

Finally, you can visualize the measures captured by the adapter just by running the diagram.



Figure 27: Real-time monitoring of 840 and 841 registers of Color Control's slave using HMI

Figure 27 shows real-time monitoring of the registers from the Color Control device. It is important to mention that those registers had values respectively of 30 and 40, but, because of the scaling applied in their mapping (Figure 23), the values were multiplied by 10.

3.2 Searching writing registers in the Color Control device to modify them in HMI OpenFMB

The register 2700 of the Color Control device sets the active power set-point of a connected Victron Quattro inverter/charger. Based on this register, the Quattro device adjusts its charging or discharging power to ensure a fixed demand.

As mentioned before, at first you need to identify the mRID of the writing profile that maps the target register. You can use the support file 'escrituraunificada-CC.xlsx' of the project repository (main branch - scripts - Esquemas de Identificacion). Remember that every writing register is associated with a ResourceDiscreteControlProfile, while all the coils use a SwitchDiscreteControlProfile. Figure 28 below illustrates how this information is searched and more.

	A	B	C	D	E	F
1	slave	mRID	Indice	Address	Scalefactor	Type
2	100	00000006-0100-1001-0000-000000000806	3	806	1	uint16
3	100	00000006-0100-1001-0000-000000000807	4	807	1	uint16
4	100	00000006-0100-1001-0000-0000000002700	5	2700	1	int16
5	100	00000006-0100-1001-0000-0000000002701	6	2701	1	uint16
6	100	00000006-0100-1001-0000-0000000002702	7	2702	1	uint16
7	100	00000006-0100-1001-0000-0000000002703	8	2703	0,01	int16
8	100	00000006-0100-1001-0000-0000000002704	9	2704	0,1	int16

Figure 28: Finding an MRID of a writing register in support files

The information from the support file for register 2700 is as follows:

- Slave number: 100
- mRID: 00000006-0100-1001-0000-0000000002700
- Indice: 5
- ScaleFactor: 1
- type: int16

This time, the support file doesn't contain the data path into the profile because all writing registers use a single ResourceDiscreteControlProfile that uses the same data path, which is 'AnOut'.

Next step is to go to HMI, and create a new mRID in 'SETTINGS - Devices', then link it with a control icon into the example diagram. Figures below show this step and more.

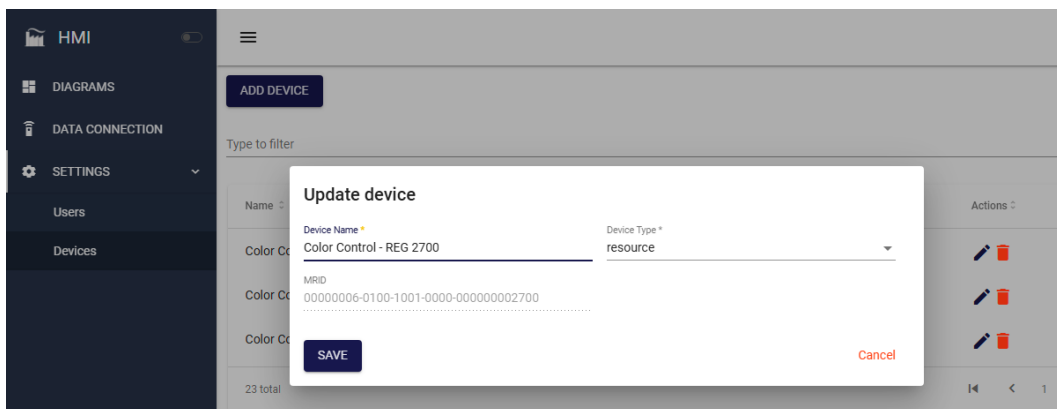


Figure 29: Registering the mRID for set-point control with OpenFMB HMI

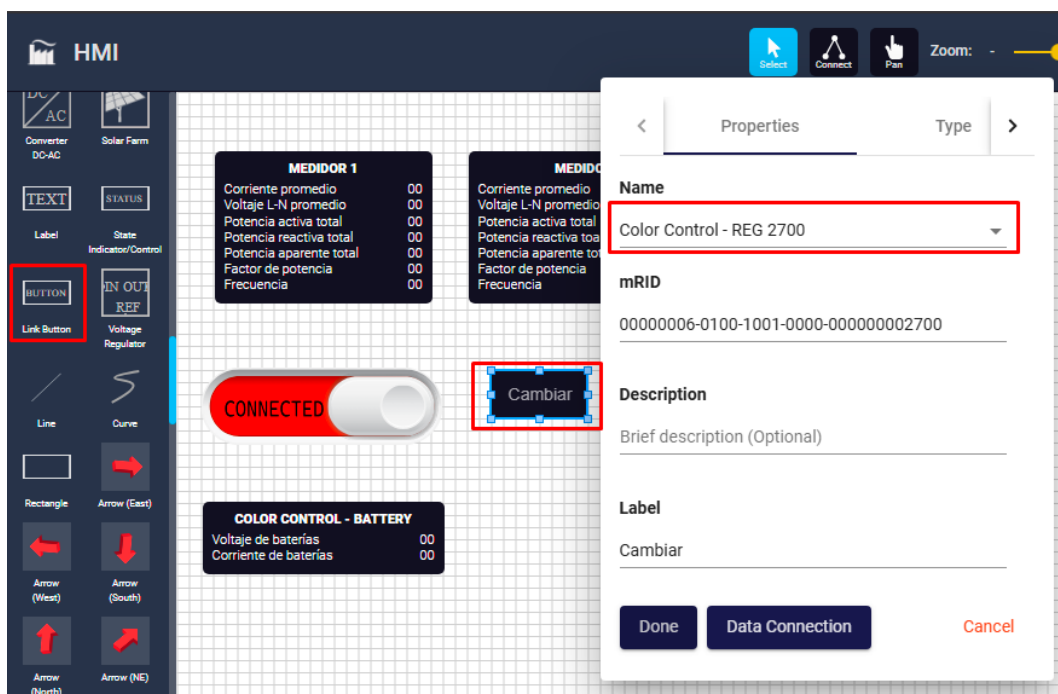


Figure 30: Creating a 'Link Button' for set-point control with HMI

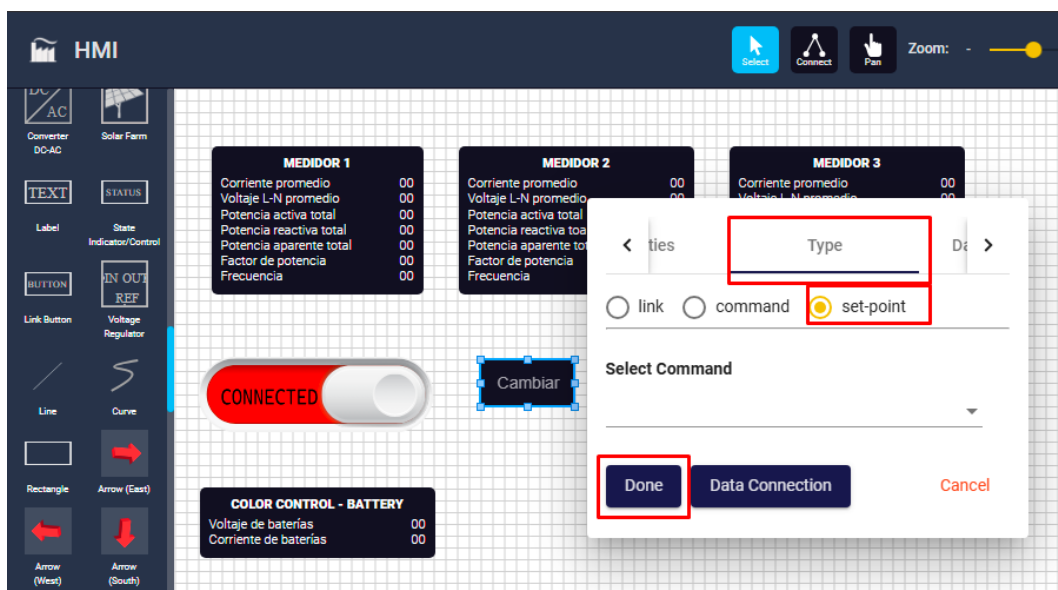


Figure 31: Changing the function of 'Link Button' to make the set-point control

Then you have the control icon linked with the mRID, and it is adjusted for set-point control; leave the 'Data Connection' configuration. In the 'Data Connection' view, you select ResourceDiscreteControlProfile and the data path is 'AnOut', then you select the only result of the filter. This step is illustrated in Figure 32.

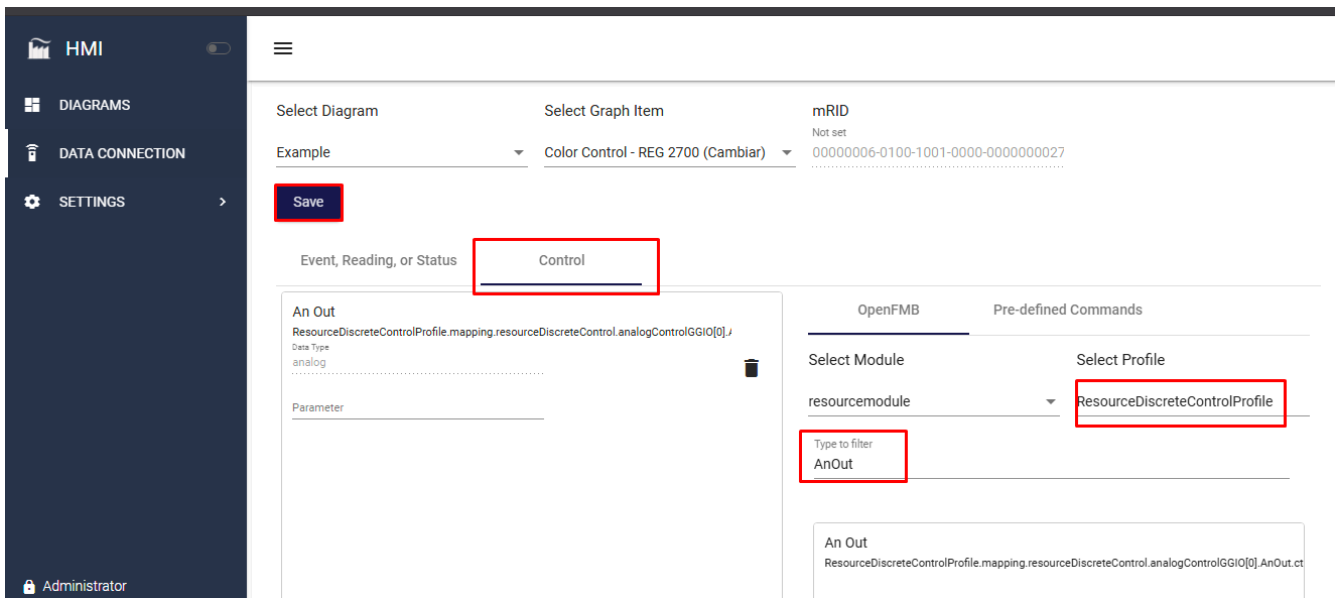


Figure 32: 'Data Connection' configuration for set-point control with HMI

Once these steps are done, the 'Link Button' already works, modifying the value of the register related to the mRID. To ensure this operation is performed correctly, we recommend that you add a MeterReadingProfile and monitor the same register; this way, you can visualize when its value is modified.

This validation can be done by using a 'Measure Box' as explained in Figures 10 to 12, at the beginning of this document. You need to add a MeterReadingProfile at the end of the file 'modbus-master-CC-slave-100.yaml' that has the mapping of the slave 100 of Color Control and maps the register 2700 as a reading register, then assign an mRID to it in the adapter's configuration. After, go to the same diagram that has the set-point control and create a 'Measure Box' for this new reading mRID, then complete the configurations in 'Data Connection' view. Finally, Figure 33 shows the use of the set-point control with diagrams of the OpenFMB HMI.

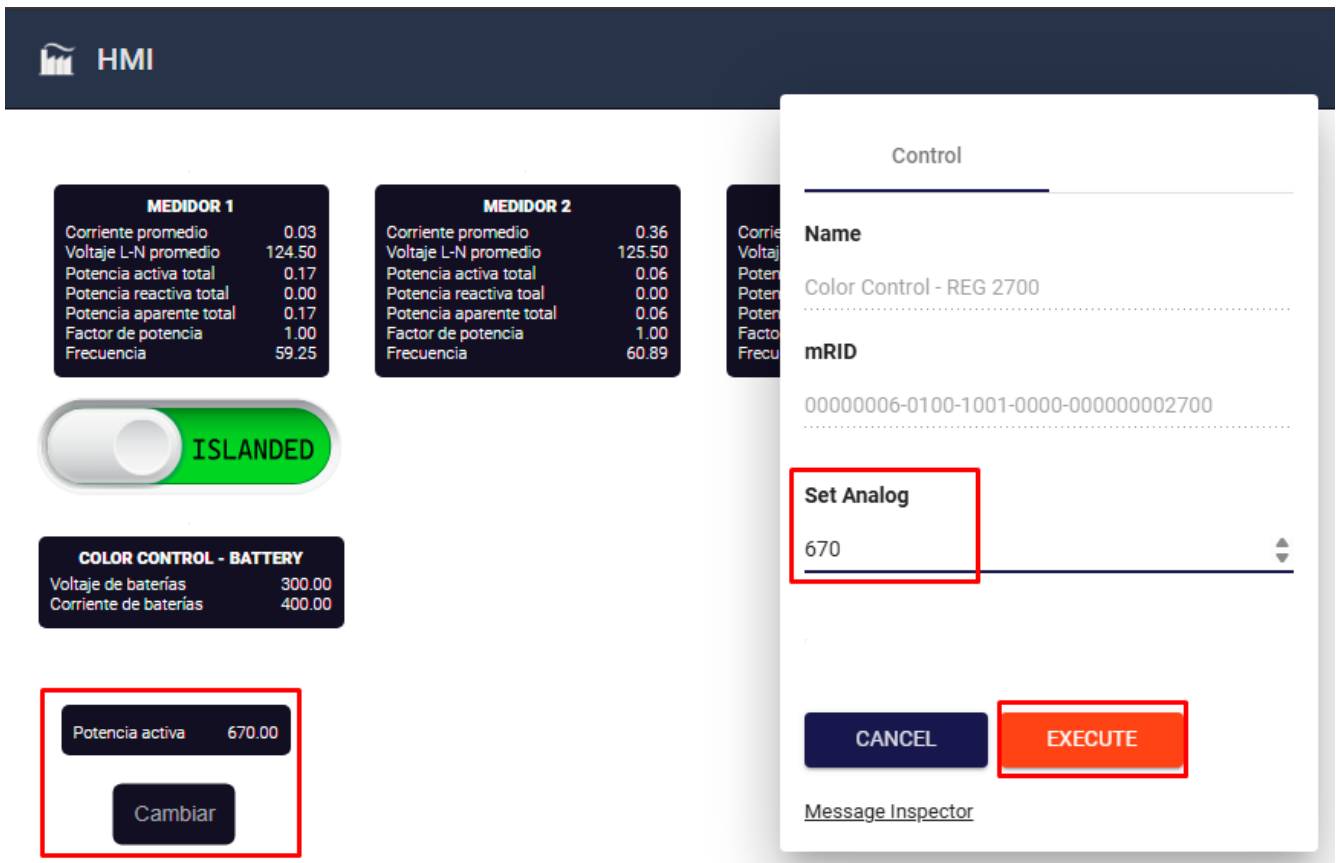


Figure 33: Control of a writing register of Color Control with HMI

4 Integrating Inverter Fronius with OpenFMB

The Fronius inverter is an IED that contains a large number of registers, all of which are mapped into its YAML file so that any register of its stack could be monitored or controlled. The complete YAML file for the Fronius inverter is named 'modbus-fronius.yaml' and is located in the project repository. Also, there is a support file in this same location for identifying the corresponding mRID and data path for any reading register of the Fronius device. The next figure shows a part of the support file for Fronius reading registers.

	A	B	C	D
1	mRID	Registro	Campo	DataPath
2	00000004-0001-0015-0000-000000000001	214	net	VA.net
3	00000004-0001-0015-0000-000000000001	500	neut	VA.neut
4	00000004-0001-0015-0000-000000000001	40001	phsA	VA.phsA
5	00000004-0001-0015-0000-000000000001	40003	phsB	VA.phsB
6	00000004-0001-0015-0000-000000000001	40004	phsC	VA.phsC
7	00000004-0001-0015-0000-000000000001	40069	net	VAr.net
8	00000004-0001-0015-0000-000000000001	40070	neut	VAr.neut
9	00000004-0001-0015-0000-000000000001	40071	phsA	VAr.phsA
10	00000004-0001-0015-0000-000000000001	40072	phsB	VAr.phsB
11	00000004-0001-0015-0000-000000000001	40074	phsC	VAr.phsC
12	00000004-0001-0015-0000-000000000001	40076	net	W.net
13	00000004-0001-0015-0000-000000000001	40078	neut	W.neut
14	00000004-0001-0015-0000-000000000001	40080	phsA	W.phsA
15	00000004-0001-0015-0000-000000000001	40082	phsB	W.phsB

Figure 34: Support file 'lectura-fronius.xlsx' for Fronius reading registers

For monitoring any Fronius register, for example 40072, you can follow the same steps specified for the Color Control Device. Summarizing these steps, at first, this section illustrates how to get the mRID and data path for a specific Fronius register using the support file, which this time is named 'lectura-fronius.xlsx'. Then you have to go to HMI and link this mRID with a name, as shown in Figure 35.

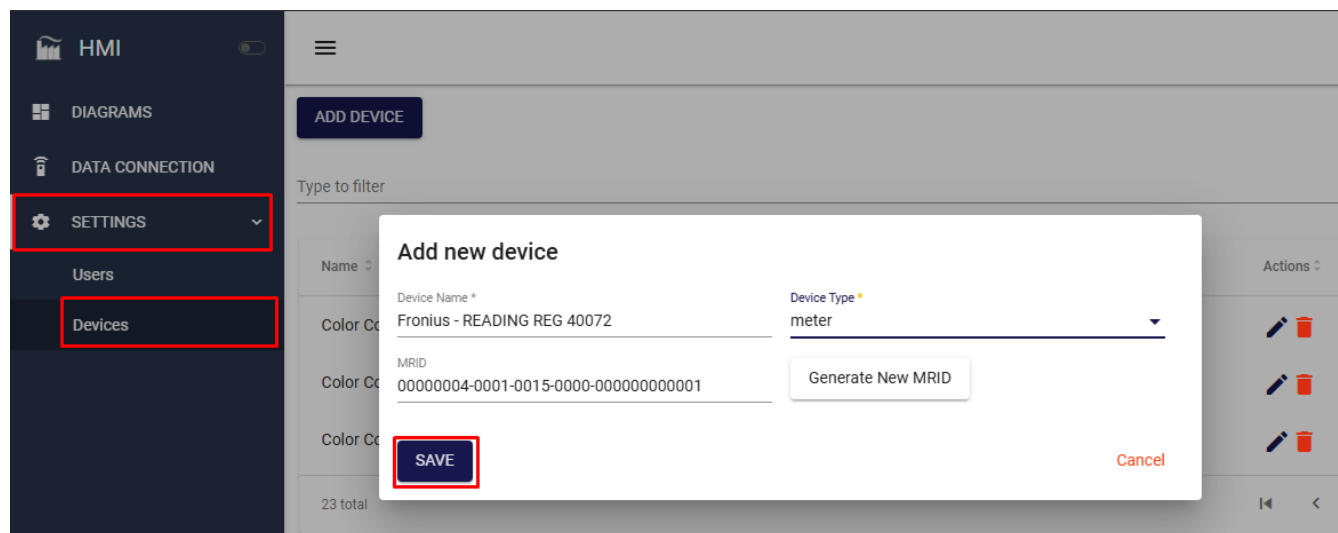


Figure 35: Registering the mRID for a Fronius reading profile in OpenFMB HMI

Then, you need to create a 'Measure Box' icon in the example diagram and link it with the mRID registered. Then, get into the 'Data Connection' view and link the 'Measure Box' added with the data path 'VAr.phsB' of a MeterReadingProfile as seen in Figure 34 for the 40072 register. These steps are shown in Figures 36 and 37.

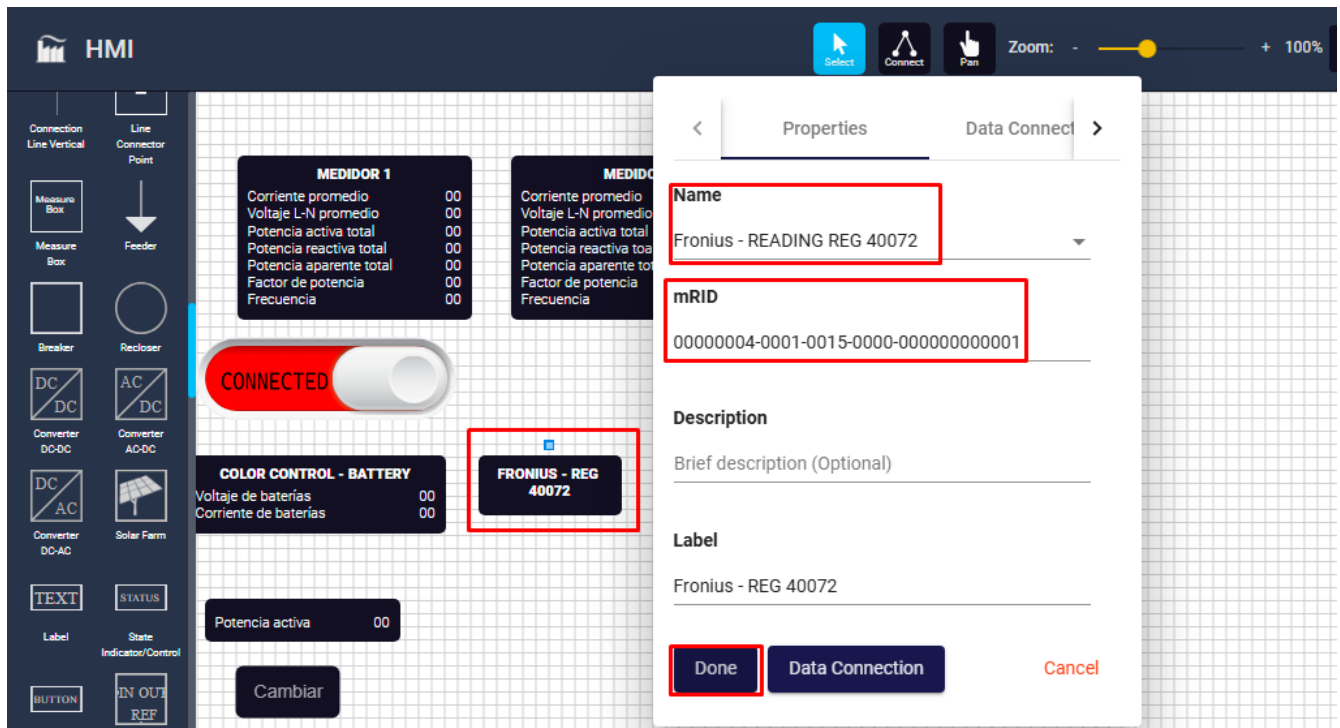


Figure 36: Linking a 'Measure Box' icon with the mRID of the Fronius reading register

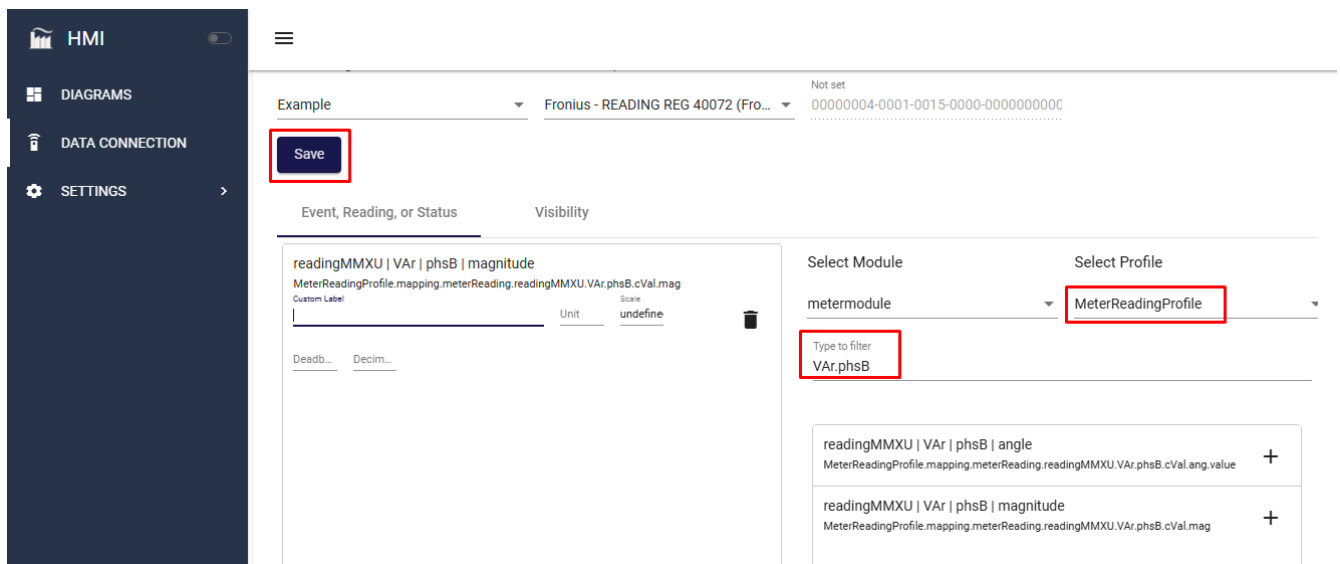


Figure 37: Configuring a 'Measure Box' icon with Fronius reading register in 'Data Connection' view

Once these configurations are done, you can execute the HMI diagram and visualize the readings of the Fronius register captured by the adapter. This visualization is illustrated in Figure 38.



Figure 38: Real-time monitoring of the register 40072 of the Fronius inverter using HMI diagrams.

For writing registers of the Fronius device, the procedure is the same as it is in the Color Control device. At first, it is important to remember that all writing registers have their own ResourceDiscreteControlProfile and their mRIDs end with the number of the register they're driving.

```

adapter.yaml X
perseo-openfmb_l > perseo-openfmb_l_27_06 > config > adapter.yaml > {} plugins > {} modbus-master > [ ]
15  plugins:
30  modbus-master:
33    sessions:
571  - path: modbus-master/modbus-fronius.yaml
574    overrides:
626      value: 00000004-0001-1001-0000-000000040167
627      - key: profiles[22].mapping.conductingEquipment.mRID.value
628        value: 00000004-0001-1001-0000-000000040240
629      - key: profiles[23].mapping.conductingEquipment.mRID.value
630        value: 00000004-0001-1001-0000-000000040241
631      - key: profiles[24].mapping.conductingEquipment.mRID.value
632        value: 00000004-0001-1001-0000-000000040242
633      #registro de limite de potencia 40243 depende del habilitador 40247
634      - key: profiles[25].mapping.conductingEquipment.mRID.value
635        value: 00000004-0001-1001-0000-004024740243
636      - key: profiles[26].mapping.conductingEquipment.mRID.value
637        value: 00000004-0001-1001-0000-000000040244
638      - key: profiles[27].mapping.conductingEquipment.mRID.value
639        value: 00000004-0001-1001-0000-000000040245
640      - key: profiles[28].mapping.conductingEquipment.mRID.value
641        value: 00000004-0001-1001-0000-000000040246
642      - key: profiles[29].mapping.protectedSwitch.conductingEquipment.mRID.value
643        value: 00000004-0001-1001-0000-000000040247
644      - key: profiles[30].mapping.conductingEquipment.mRID.value
645        value: 00000004-0001-1001-0000-000000040248
646      - key: profiles[31].mapping.conductingEquipment.mRID.value
647        value: 00000004-0001-1001-0000-000000040249
648      - key: profiles[32].mapping.conductingEquipment.mRID.value

```

Figure 39: Adapter configurations with mRIDs for Fronius Writing registers

For controlling a Fronius register with HMI, you need to add the mRID in HMI using the view ‘SETTINGS - Devices’, then create a ‘Link Button’ in a diagram and change its operation to set-point (Figures 30 and 31). Once it is done, go to the ‘Data Connection’ view to add the data path of the writing register to the ‘Link Button’. This data path, as mentioned before, is ‘AnOut’ for a ResourceDiscreteControlProfile. Finally, you can modify the registers of the Fronius device using the control icons of the HMI diagrams.

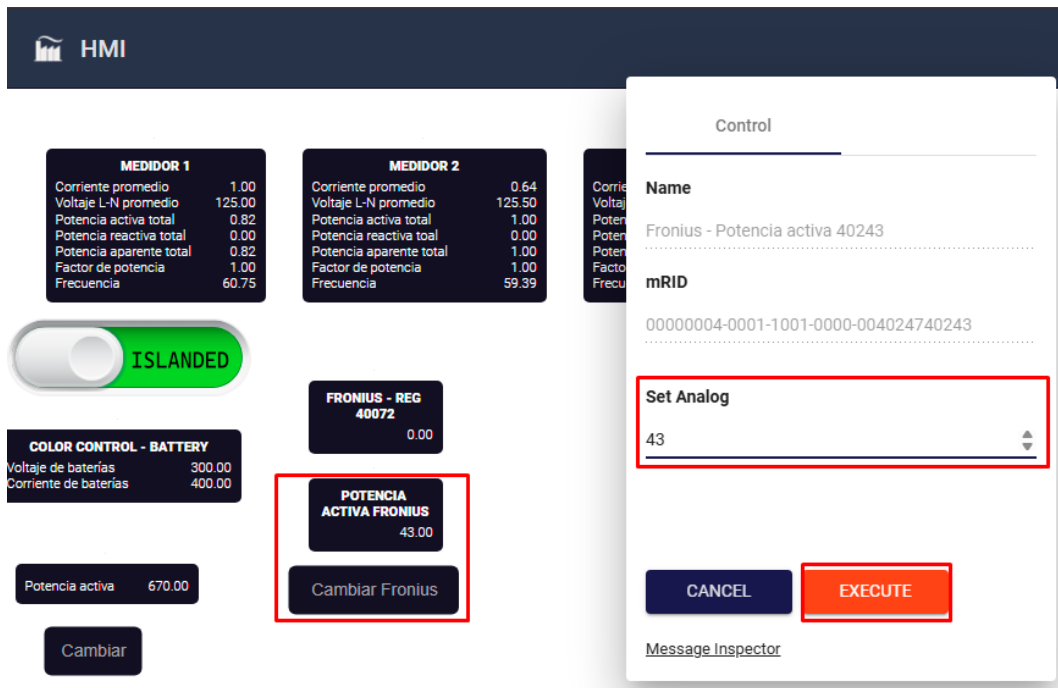


Figure 40: Modifying a register of the Fronius device with HMI diagrams

There is an important clarification at this point. One of the relevant writing registers of Fronius is 40243 that adjusts its active power limit; however, this register is enabled by another writing register with index 40247, so if this last register has a zero value, then register 40243 can't be changed. To achieve this joint operation, an external development in HMI was required, allowing it to use the mRID codification for implementing the joint operation between registers. The details about this development are not necessary for now, but this operation is shown as part of the example of the use of the writing registers of Fronius. The figures below correspond to the main control diagram of UdeA's microgrid.

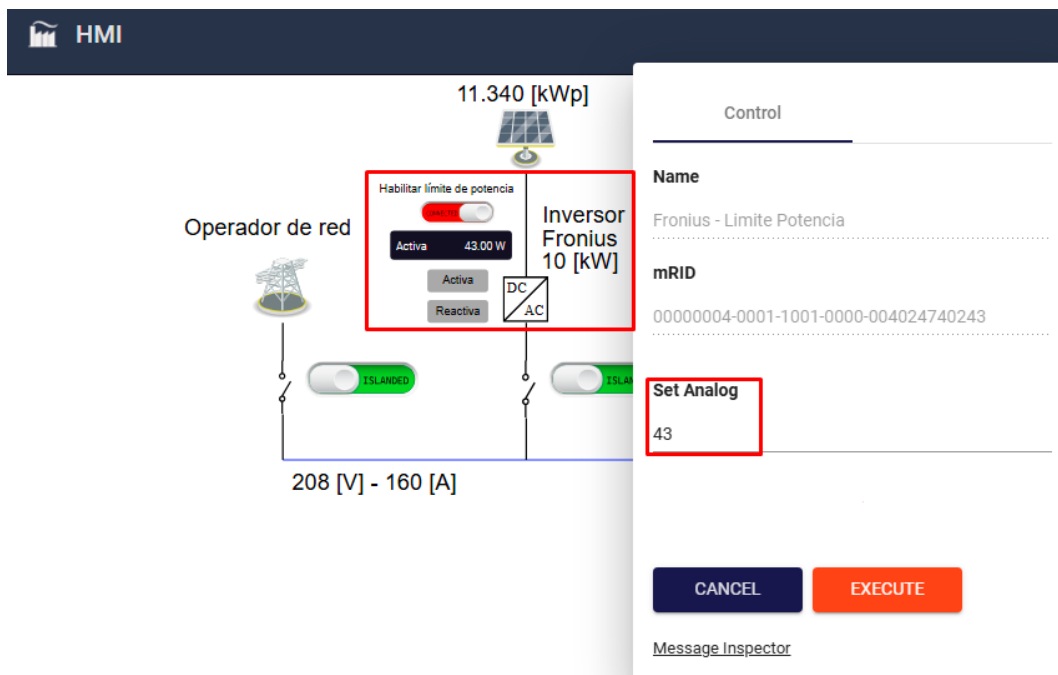


Figure 41: Modifying active power limit of Fronius by joint operation with register 40247



Figure 42: Disabling set-point adjust of Fronius active power limit by joint operation with register 40247